Comprehensive Testing Plan

Overview

This document outlines all unit and E2E tests needed for the OFSP backend modules and helper services.

Test Coverage Status

✅ Existing Tests

- **Notification Module**: service.spec.ts, controller.spec.ts, e2e-spec.ts
- **Payment Module**: service.spec.ts, controller.spec.ts, e2e-spec.ts
- **Marketplace Module**: service.spec.ts, controller.spec.ts, e2e-spec.ts
- **Profile Module**: service.spec.ts, controller.spec.ts, e2e-spec.ts
- **Input Module**: service.spec.ts, controller.spec.ts
- **Transport Module**: service.spec.ts, controller.spec.ts

❌ Missing Tests

- **Auth Module**: All tests missing
- **Users Module**: All tests missing
- **Aggregation Module**: All tests missing
- **Helper Services**: NotificationHelperService, ActivityLogService
- **Utilities**: Traceability utilities
- **E2E Tests**: Auth, Users, Input, Transport, Aggregation

Testing Todo List

1. Auth Module Tests

Unit Tests - Auth Service (`auth.service.spec.ts`)

- [ ] **register()**
  - [ ] Should register new user successfully
  - [ ] Should hash password before storing
  - [ ] Should create profile with user
  - [ ] Should throw BadRequestException if email exists
  - [ ] Should throw BadRequestException if phone exists
  - [ ] Should set status to PENDING_VERIFICATION
  - [ ] Should generate JWT tokens on registration
- [ ] **login()**
  - [ ] Should login with valid credentials
  - [ ] Should return access and refresh tokens
  - [ ] Should throw UnauthorizedException for invalid email
  - [ ] Should throw UnauthorizedException for invalid password
  - [ ] Should throw UnauthorizedException for inactive user
  - [ ] Should update lastLoginAt timestamp
- [ ] **validateUser()**
  - [ ] Should return user if credentials valid
  - [ ] Should throw UnauthorizedException if user not found
  - [ ] Should throw UnauthorizedException if password invalid
- [ ] **refreshToken()**
  - [ ] Should generate new access token with valid refresh token
  - [ ] Should throw UnauthorizedException for invalid refresh token
  - [ ] Should throw UnauthorizedException for expired refresh token
- [ ] **verifyEmail()**
  - [ ] Should verify email and update status
  - [ ] Should throw BadRequestException for invalid token
  - [ ] Should throw BadRequestException for expired token

Unit Tests - Auth Controller (`auth.controller.spec.ts`)

- [ ] **POST /auth/register**
  - [ ] Should register new user

- [ ] Should return 400 for duplicate email
- [ ] Should validate DTO fields

- [ ] **POST /auth/login**
  - [ ] Should login successfully
  - [ ] Should return 401 for invalid credentials
  - [ ] Should return tokens in response
- [ ] **POST /auth/refresh**
  - [ ] Should refresh access token
  - [ ] Should return 401 for invalid refresh token
- [ ] **POST /auth/verify-email**
  - [ ] Should verify email
  - [ ] Should return 400 for invalid token

E2E Tests - Auth (`test/auth.e2e-spec.ts`)

- [ ] **POST /auth/register**
  - [ ] Should create user and profile
  - [ ] Should return tokens
  - [ ] Should reject duplicate email
  - [ ] Should reject duplicate phone
- [ ] **POST /auth/login**
  - [ ] Should login with correct credentials
  - [ ] Should reject incorrect password
  - [ ] Should reject non-existent email
- [ ] **POST /auth/refresh**
  - [ ] Should refresh access token
  - [ ] Should reject invalid refresh token
- [ ] **POST /auth/verify-email**
  - [ ] Should verify email successfully
  - [ ] Should reject invalid token
- [ ] **POST /auth/logout**
  - [ ] Should invalidate refresh token

2. Users Module Tests

Unit Tests - Users Service (`users.service.spec.ts`)

- [ ] **findById()**
  - [ ] Should return user with profile
  - [ ] Should omit password from response
  - [ ] Should throw NotFoundException if user not found
- [ ] **findByEmail()**
  - [ ] Should return user with profile
  - [ ] Should omit password from response
  - [ ] Should return null if user not found
- [ ] **updateProfile()**
  - [ ] Should update profile successfully
  - [ ] Should throw NotFoundException if profile not found

Unit Tests - Users Controller (`users.controller.spec.ts`)

- [ ] **GET /users/:id**
  - [ ] Should return user by ID
  - [ ] Should return 404 if not found
  - [ ] Should omit password
- [ ] **GET /users/email/:email**
  - [ ] Should return user by email
  - [ ] Should return 404 if not found

E2E Tests - Users (`test/users.e2e-spec.ts`)

- [ ] **GET /users/:id**
  - [ ] Should return user details
  - [ ] Should require authentication
  - [ ] Should return 404 for non-existent user
- [ ] **GET /users/email/:email**
  - [ ] Should return user by email
  - [ ] Should require authentication

3. Profile Module Tests (Enhancement)

Unit Tests - Profile Service (Enhance existing)

- [ ] **Additional tests for:**
  - [ ] Rating creation with validation
  - [ ] Rating summary calculations
  - [ ] Profile update with role-based restrictions

E2E Tests - Profile (Enhance existing)

- [ ] **Additional tests for:**
  - [ ] Complete rating flow
  - [ ] Rating aggregation accuracy
  - [ ] Profile update permissions

4. Marketplace Module Tests (Enhancement)

Unit Tests - Marketplace Service (Enhance existing)

- [ ] **Additional tests for lifecycle compliance:**
  - [ ] createOrder() - batchId and QR code generation
  - [ ] createOrder() - notification creation
  - [ ] createOrder() - activity log creation
  - [ ] createOrder() - negotiation status update
  - [ ] createOrder() - RFQ response status update
  - [ ] updateOrderStatus() - status transition validation
  - [ ] updateOrderStatus() - status-specific logic
  - [ ] updateOrderStatus() - notification creation
  - [ ] updateOrderStatus() - activity log creation
  - [ ] updateOrderStatus() - status history update

E2E Tests - Marketplace (Enhance existing)

- [ ] **Complete order lifecycle flow:**
  - [ ] Order creation → notifications sent
  - [ ] Order status transitions with validation
  - [ ] Payment secured → order status update
  - [ ] Transport accepted → order status update

- [ ] Stock in → order status update
- [ ] Quality check → order status update
- [ ] Delivery → order status update

- [ ] **Negotiation to order conversion:**
  - [ ] Create order from negotiation
  - [ ] Verify negotiation status updated
- [ ] **RFQ to order conversion:**
  - [ ] Create order from RFQ response
  - [ ] Verify RFQ response status updated

5. Input Module Tests (Enhancement)

Unit Tests - Input Service (Enhance existing)

- [ ] **Additional tests for lifecycle compliance:**
  - [ ] createInputOrder() - notification creation
  - [ ] createInputOrder() - activity log creation
  - [ ] updateInputOrderStatus() - notification creation
  - [ ] updateInputOrderStatus() - activity log creation
  - [ ] updateInputOrderStatus() - stock reduction on acceptance

E2E Tests - Input (`test/input.e2e-spec.ts`)

- [ ] **Input Products:**
  - [ ] Create input product
  - [ ] Update input product
  - [ ] Delete input product
  - [ ] List inputs with filters
- [ ] **Input Orders:**
  - [ ] Create input order → notifications sent
  - [ ] Update order status → notifications sent
  - [ ] Complete order lifecycle
  - [ ] Stock reduction on acceptance
- [ ] **Input Customers:**
  - [ ] Get customer list
  - [ ] Get customer order history

6. Transport Module Tests (Enhancement)

Unit Tests - Transport Service (Enhance existing)

- [ ] **Additional tests for lifecycle compliance:**
  - [ ] createTransportRequest() - activity log creation
  - [ ] updateTransportRequestStatus() - notification creation
  - [ ] updateTransportRequestStatus() - activity log creation
  - [ ] updateTransportRequestStatus() - order status update (when linked)
  - [ ] acceptTransportRequest() - notification creation
  - [ ] addTrackingUpdate() - activity log creation

E2E Tests - Transport (`test/transport.e2e-spec.ts`)

- [ ] **Transport Requests:**
  - [ ] Create transport request → activity log created
  - [ ] Accept request → notifications sent
  - [ ] Update status → notifications sent
  - [ ] Update status → order status updated (if linked)
  - [ ] Add tracking updates
  - [ ] Complete delivery flow
- [ ] **Pickup Schedules:**
  - [ ] Create pickup schedule
  - [ ] Book pickup slot
  - [ ] Complete pickup flow
- [ ] **Integration with Orders:**
  - [ ] Transport accepted → order IN_TRANSIT
  - [ ] Transport delivered → order DELIVERED

7. Aggregation Module Tests

Unit Tests - Aggregation Service (`aggregation.service.spec.ts`)

- [ ] **Aggregation Centers:**
  - [ ] getAggregationCenters() - with filters
  - [ ] getAggregationCenterById() - with relations
  - [ ] createAggregationCenter() - code generation
  - [ ] updateAggregationCenter() - all fields

- [ ] **Stock Transactions:**
  - [ ] getStockTransactions() - with filters and farmer relation
  - [ ] createStockIn() - farmer info derivation from order
  - [ ] createStockIn() - activity log creation
  - [ ] createStockIn() - order status update
  - [ ] createStockOut() - order status update
  - [ ] createStockOut() - activity log creation
- [ ] **Inventory:**
  - [ ] getInventory() - with farmer relation
  - [ ] Inventory filtering by center
- [ ] **Quality Checks:**
  - [ ] getQualityChecks() - with filters
  - [ ] createQualityCheck() - farmer info derivation
  - [ ] createQualityCheck() - batchId derivation
  - [ ] createQualityCheck() - notification creation
  - [ ] createQualityCheck() - activity log creation
  - [ ] createQualityCheck() - order status update (QUALITY_CHECKED → QUALITY_APPROVED/REJECTED)
  - [ ] createQualityCheck() - order qualityScore update
- [ ] **Wastage:**
  - [ ] getWastageEntries() - with farmer relation
  - [ ] createWastageEntry() - farmer info derivation from inventory
  - [ ] createWastageEntry() - batchId derivation
  - [ ] createWastageEntry() - recordedByName population
- [ ] **Statistics:**
  - [ ] getAggregationStats() - all metrics
  - [ ] Null handling for aggregates

Unit Tests - Aggregation Controller (`aggregation.controller.spec.ts`)

- [ ] **GET /aggregation/centers**
  - [ ] Should return centers with filters
- [ ] **POST /aggregation/centers**
  - [ ] Should create center
- [ ] **GET /aggregation/transactions**
  - [ ] Should return transactions
- [ ] **POST /aggregation/transactions/stock-in**
  - [ ] Should create stock in
- [ ] **POST /aggregation/transactions/stock-out**

- [ ] Should create stock out
- [ ] **GET /aggregation/inventory**
  - [ ] Should return inventory
- [ ] **GET /aggregation/quality-checks**
  - [ ] Should return quality checks
- [ ] **POST /aggregation/quality-checks**
  - [ ] Should create quality check
- [ ] **GET /aggregation/wastage**
  - [ ] Should return wastage entries
- [ ] **POST /aggregation/wastage**
  - [ ] Should create wastage entry
- [ ] **GET /aggregation/stats**
  - [ ] Should return statistics

E2E Tests - Aggregation (`test/aggregation.e2e-spec.ts`)

- [ ] **Aggregation Centers:**
  - [ ] Create center
  - [ ] Update center
  - [ ] Get center with inventory
- [ ] **Stock Transactions:**
  - [ ] Stock in → order status AT_AGGREGATION
  - [ ] Stock in → activity log created
  - [ ] Stock out → order status OUT_FOR_DELIVERY
  - [ ] Stock out → activity log created
  - [ ] Farmer traceability maintained
- [ ] **Quality Checks:**
  - [ ] Create quality check → order QUALITY_CHECKED
  - [ ] Quality approved → order QUALITY_APPROVED
  - [ ] Quality rejected → order QUALITY_REJECTED
  - [ ] Quality check → notifications sent
  - [ ] Quality check → activity log created
  - [ ] Quality check → order qualityScore updated
- [ ] **Wastage:**
  - [ ] Create wastage entry
  - [ ] Farmer info derived from inventory
- [ ] **Statistics:**
  - [ ] Get aggregation statistics

8. Payment Module Tests (Enhancement)

Unit Tests - Payment Service (Enhance existing)

- [ ] **Additional tests for lifecycle compliance:**
  - [ ] updatePaymentStatus() - order status update when SECURED
  - [ ] updatePaymentStatus() - notification creation
  - [ ] updatePaymentStatus() - activity log creation
  - [ ] releaseEscrow() - notification creation
  - [ ] disputeEscrow() - notification creation

E2E Tests - Payment (Enhance existing)

- [ ] **Payment Flow:**
  - [ ] Create payment
  - [ ] Update payment to SECURED → order status PAYMENT_SECURED
  - [ ] Update payment to RELEASED → notifications sent
- [ ] **Escrow Flow:**
  - [ ] Create escrow
  - [ ] Release escrow → notifications sent
  - [ ] Dispute escrow → notifications sent
- [ ] **Integration with Orders:**
  - [ ] Payment secured → order status update
  - [ ] Payment released → order completion

9. Helper Services Tests

Unit Tests - NotificationHelperService (`common/services/notification-helper.service.spec.ts`)

- [ ] **createNotification()**
  - [ ] Should create single notification
  - [ ] Should set default priority to MEDIUM
  - [ ] Should handle all notification fields
- [ ] **createNotifications()**
  - [ ] Should create multiple notifications
  - [ ] Should handle empty array

- [ ] **notifyOrderPlaced()**
  - [ ] Should create notifications for farmer and buyer
  - [ ] Should set correct priorities
  - [ ] Should include order metadata
- [ ] **notifyOrderStatusChange()**
  - [ ] Should create notifications for buyer and farmer
  - [ ] Should use correct status messages
  - [ ] Should set priorities based on status
  - [ ] Should handle all status types
  - [ ] Should include actionUrl and actionLabel

Unit Tests - ActivityLogService (`common/services/activity-log.service.spec.ts`)

- [ ] **createActivityLog()**
  - [ ] Should create activity log
  - [ ] Should handle optional fields
  - [ ] Should store metadata as JSON
- [ ] **createActivityLogs()**
  - [ ] Should create multiple logs
  - [ ] Should handle empty array
- [ ] **logOrderCreated()**
  - [ ] Should create log with correct action
  - [ ] Should include order metadata
- [ ] **logOrderStatusChange()**
  - [ ] Should create log with old and new status
  - [ ] Should include order metadata
- [ ] **logPaymentCreated()**
  - [ ] Should create log with payment metadata
- [ ] **logTransportCreated()**
  - [ ] Should create log with transport metadata

Unit Tests - Traceability Utilities (`common/utils/traceability.util.spec.ts`)

- [ ] **generateBatchId()**
  - [ ] Should generate unique batch IDs
  - [ ] Should follow format BATCH-YYYYMMDD-HHMMSS-XXXXXX
  - [ ] Should generate different IDs on each call
- [ ] **generateQRCode()**

- [ ] Should generate QR code from batchId
- [ ] Should follow format QR-{batchId}

- [ ] **generateBatchTraceability()**
  - [ ] Should return both batchId and qrCode
  - [ ] Should have matching batchId in QR code

10. Integration Tests

Cross-Service Integration Tests

- [ ] **Payment → Marketplace Order:**
  - [ ] Payment secured → order status updated
  - [ ] Payment secured → notifications sent
- [ ] **Transport → Marketplace Order:**
  - [ ] Transport IN_TRANSIT → order IN_TRANSIT
  - [ ] Transport DELIVERED → order DELIVERED
- [ ] **Aggregation → Marketplace Order:**
  - [ ] Stock in → order AT_AGGREGATION
  - [ ] Quality check → order QUALITY_CHECKED → QUALITY_APPROVED/REJECTED
  - [ ] Stock out → order OUT_FOR_DELIVERY

Complete Lifecycle E2E Tests

- [ ] **Full Order Lifecycle:**
  - [ ] Order created → notifications → activity logs
  - [ ] Order accepted → escrow created
  - [ ] Payment secured → order status update → notifications
  - [ ] Transport accepted → order IN_TRANSIT → notifications
  - [ ] Stock in → order AT_AGGREGATION → notifications
  - [ ] Quality check → order QUALITY_APPROVED → notifications
  - [ ] Stock out → order OUT_FOR_DELIVERY → notifications
  - [ ] Transport delivered → order DELIVERED → notifications
  - [ ] Payment released → order COMPLETED → notifications
  - [ ] Verify all notifications created
  - [ ] Verify all activity logs created
  - [ ] Verify batchId traceability throughout

Test File Structure

```
src/
├── modules/
│   ├── auth/
│   │   ├── auth.service.spec.ts          [NEW]
│   │   └── auth.controller.spec.ts        [NEW]
│   ├── users/
│   │   ├── users.service.spec.ts          [NEW]
│   │   └── users.controller.spec.ts       [NEW]
│   ├── aggregation/
│   │   ├── aggregation.service.spec.ts    [NEW]
│   │   └── aggregation.controller.spec.ts [NEW]
│   └── [other modules - enhance existing]
├── common/
│   ├── services/
│   │   ├── notification-helper.service.spec.ts [NEW]
│   │   └── activity-log.service.spec.ts  [NEW]
│   └── utils/
│       └── traceability.util.spec.ts     [NEW]

test/
├── auth.e2e-spec.ts                       [NEW]
├── users.e2e-spec.ts                      [NEW]
├── input.e2e-spec.ts                      [NEW]
├── transport.e2e-spec.ts                  [NEW]
├── aggregation.e2e-spec.ts               [NEW]
├── payment.e2e-spec.ts                    [ENHANCE]
├── marketplace.e2e-spec.ts                [ENHANCE]
└── profile.e2e-spec.ts                    [ENHANCE]
```

Testing Priorities

High Priority (Critical Functionality)

1. Auth Module - Authentication is critical
2. Marketplace Module enhancements - Order lifecycle compliance
3. Aggregation Module - Stock and quality check flows

4. Payment Module enhancements - Order status integration
5. Helper Services - Used by all modules

Medium Priority (Important Features)

6. Users Module - User management
7. Transport Module enhancements - Order status integration
8. Input Module enhancements - Notifications and logs
9. Complete lifecycle E2E tests

Low Priority (Nice to Have)

10. Profile Module enhancements
11. Additional edge case tests
12. Performance tests

Test Implementation Guidelines

Unit Test Structure

```
describe('ServiceName', () => {
  describe('methodName()', () => {
    it('should [expected behavior]', async () => {
      // Arrange
      // Act
      // Assert
    });

    it('should throw [Exception] when [condition]', async () => {
      // Test error cases
    });
  });
});
```

E2E Test Structure

```
describe('ModuleName (e2e)', () => {
  let app: INestApplication;
  let prisma: PrismaService;
  let authToken: string;

  beforeAll(async () => {
    // Setup app and test data
  });

  afterAll(async () => {
    // Cleanup
  });

  describe('POST /endpoint', () => {
    it('should [expected behavior]', async () => {
      // Test with real HTTP requests
    });
  });
});
```

Mocking Guidelines

- Mock PrismaService for unit tests
- Use real database for E2E tests (test database)
- Mock external services (payment gateways, etc.)
- Use test utilities for common mocks

Estimated Test Count

Unit Tests

- Auth Module: ~25 tests
- Users Module: ~10 tests
- Aggregation Module: ~30 tests
- Helper Services: ~20 tests
- Utilities: ~5 tests

- **Enhancements to existing**: ~30 tests
- **Total: ~120 new unit tests**

E2E Tests

- Auth Module: ~10 tests
- Users Module: ~5 tests
- Aggregation Module: ~15 tests
- Input Module: ~10 tests
- Transport Module: ~10 tests
- **Enhancements to existing**: ~15 tests
- **Total: ~65 new E2E tests**

Grand Total: ~185 new tests

Notes

- All tests should follow existing patterns in the codebase
- Use test utilities from `src/test/test-utils.ts` where available
- E2E tests should use `setupTestApp`, `createTestUser`, `getAuthToken` helpers
- Ensure tests clean up after themselves
- Mock external dependencies appropriately
- Test both success and error cases
- Test edge cases and boundary conditions